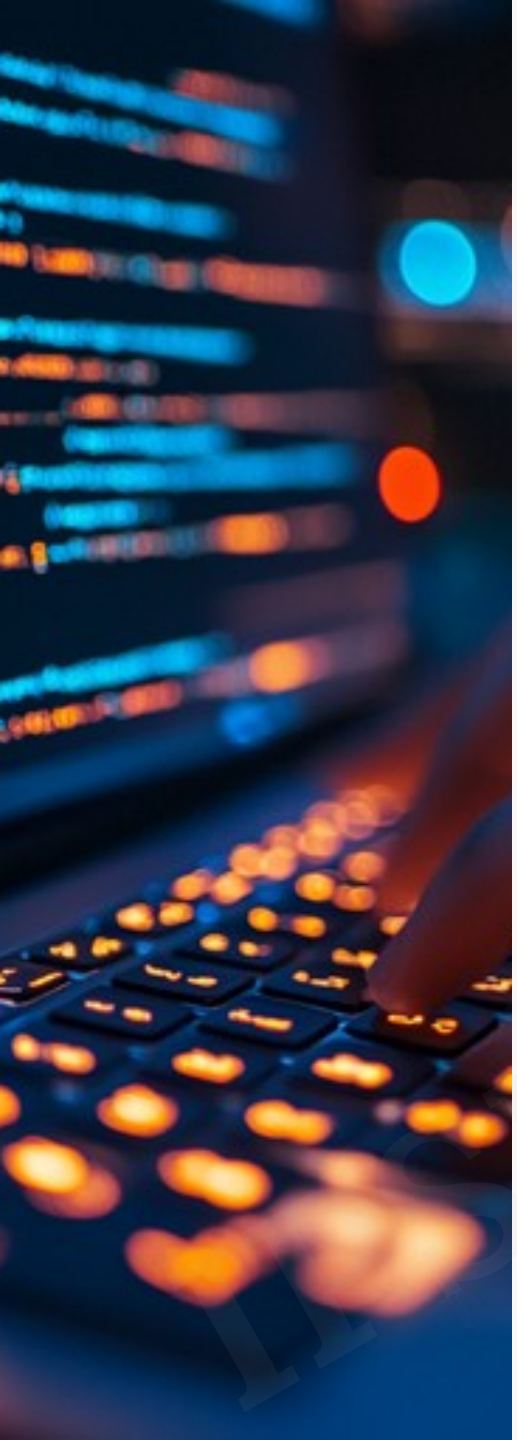


MSYS2

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

MSYS2 Introduction

MinGW

- Minimalist GNU for Windows（GNU 工具鏈在 Windows 上的移植版）

版本

- MinGW
  - 這個工具鏈能夠只能產生 32 位元 Windows 程式，對應 Win32 API。
- MinGW-w64
  - 從原始 MinGW 專案分支出來的新版。
  - MinGW 的「for Win64」版本。
  - 這個工具鏈能夠能產生 32 位元 (Win32) 與 64 位元 (Win64) 程式，支援更多 API、C++ 標準與執行期。

IPSI Lab, Yuntech-CSIE

MSYS2 用途與特點

1. **提供 Linux 類似的開發環境**
   - 支援常見的 Linux 命令與工具，例如 `bash`、`ls`、`grep` 等。
   - 讓你在 Windows 平台上能夠像在 Linux 一樣進行開發工作。
2. **支援 MinGW 編譯器**
   - MSYS2 支援 **MinGW-w64** 編譯器，讓你能夠在 Windows 上編譯並生成原生的 Windows 應用程式。
3. **Pacman 套件管理器**
   - MSYS2 使用 **Pacman** 套件管理器來安裝、更新與移除軟體套件，讓你能輕鬆地管理開發工具。

IPSI Lab, Yuntech-CSIE

MSYS2 的主要組成

1. **MSYS2 Shell**
   - 提供類似 Unix 的終端環境，支援 bash shell 和 GNU 工具。
2. **MinGW-w64**
   - 讓開發者能夠使用 `gcc` 或 `clang` 編譯器，生成可在 Windows 上執行的原生應用程式。
3. **Pacman**
   - 一個快速、高效的套件管理器，負責管理系統中的軟體套件和依賴性。

IPSI Lab, Yuntech-CSIE

MSYS2 的應用場景

1. **跨平台開發**
   - 適合需要在 Windows 上模擬或進行 Linux 開發的場景。
2. **開發 Windows 原生應用程式**
   - 使用 MinGW 編譯器編譯 C/C++ 程式，生成 Windows 上的可執行檔。
3. **使用開源工具**
   - 可以輕鬆安裝和使用如 Git、Python、Node.js 等開源工具，並在 Windows 上運行。

IPSI Lab, Yuntech-CSIE

MSYS2 vs 傳統開發環境

特性	MSYS2	傳統 Windows 開發環境
套件管理	使用 Pacman 進行套件管理	需要手動下載與安裝
終端工具	提供 Linux 類似的命令行環境	只支援 cmd 或 PowerShell
編譯器支援	支援 MinGW、Clang 編譯器	多需單獨安裝

IPSI Lab, Yuntech-CSIE

MSYS2 的所有 GCC 編譯器版本

- `mingw-w64-ucrt-x86_64-gcc`（64 位元，UCRT）
- `mingw-w64-ucrt-i686-gcc`（32 位元，UCRT）
- `mingw-w64-x86_64-gcc`（64 位元，MSVCRT）
- `mingw-w64-i686-gcc`（32 位元，MSVCRT）

IPSI Lab, Yuntech-CSIE

另有SEH（Structured Exception Handling）與 DWARF（Debug With Arbitrary Record Format），以及其他的版本。

- `mingw-w64-x86_64-gcc` (SEH)（64 位元，MSVCRT）
- `mingw-w64-i686-gcc` (DWARF)（32 位元，MSVCRT）

[NOTE]

From: https://learn.microsoft.com/en-us/cpp/cpp/structured-exception-handling-c-cpp?view=msvc-170

- Structured exception handling (SEH) is a Microsoft extension to C and C++ to handle certain exceptional code situations, such as hardware faults, gracefully.
- Although Windows and Microsoft C++ support SEH, we recommend that you use ISO-standard C++ exception handling in C++ code. It makes your code more portable and flexible.

CPU 指令集架構（ISA） 的位元寬度

- 64 位元架構：`x86_64`
- 32 位元架構：`i686`

項目	`x86_64`	`i686`
位元寬度	64 位元	32 位元
記憶體存取上限	超過 4GB（理論上可達 TB）	約 4GB
指令集相容性	可執行 64 位與 32 位程式	僅能執行 32 位程式
常見用途	現代 PC、伺服器	舊系統、輕量環境
代表意義	“x86 的 64 位延伸”	“第六代 32 位 x86 CPU”

IPSI Lab, Yuntech-CSIE

UCRT (Universal C Runtime)

- 這個版本的 `gcc` 使用的是 Universal C Runtime，它是 Windows 10 之後的標準 C Runtime 庫。
- UCRT 是 Microsoft 推出的標準 C Runtime 庫，並且隨著 Windows 10 的更新來分發。
- 適用於 Windows 10 及以上的系統，因為這些系統內建 UCRT，能夠更好地支援現代化應用程式開發。
- 同時有64 位元 (x86_64)與32 位元 (i686)版本。

IPSI Lab, Yuntech-CSIE

MSVCRT (Microsoft C Runtime)

- 這個版本使用的是舊的 Microsoft C Runtime（MSVCRT）。
- 這是 Windows 95 以來一直使用的標準 C Runtime。
- 儘管功能較老，但它在 Windows 各個版本中有較廣泛的支援，從舊版 Windows（如 Windows 7）到新版 Windows 都能運行。
- 適合需要向後相容的應用程式開發。
  - 如果你的應用程式需要支援 Windows 7、Windows 8 等較舊版本的 Windows，這個版本的 `gcc` 會比較適合。
- 同時有64 位元 (x86_64)與32 位元 (i686)版本。

IPSI Lab, Yuntech-CSIE

Icon & Package Naming

	Name	Package prefix
	**MSYS**	None
	**MINGW64**	`mingw-w64-x86_64-`
	**UCRT64**	`mingw-w64-ucrt-x86_64-`
	**CLANG64**	`mingw-w64-clang-x86_64-`

Mintty is the default terminal application in MSYS2.

	Name	Package prefix
	MINGW32	`mingw-w64-i686-`
	CLANG32	`mingw-w64-clang-i686-`
	CLANGARM64	`mingw-w64-clang-aarch64-`

IPSI Lab, Yuntech-CSIE

Package Management (`pacman`)

Finding a package

```
pacman -Ss <name or part of the name of the package>
```

Example

```
$ pacman -Ss openjp
```

```
mingw64/mingw-w64-x86_64-openjpeg 1.5.2-7
    An open source JPEG 2000 codec (mingw-w64)
mingw64/mingw-w64-x86_64-openjpeg2 2.1.0-7 [installed]
    An open source JPEG 2000 codec (mingw-w64)
```

ps. mingw-w64-x86_64-openjpeg2 package is installed,

Search Installed Packages Only

```
pacman -Qs <name or part of the name of the package>
```

Installing a package

```
pacman -S <name of the package>
```

Uninstalling a package

```
pacman -R <name of the package>
```

IPSI Lab, yuntech-CSIE

Finding dependencies of a package

Method 1. (`pactree`)

```
$ pactree mingw-w64-x86_64-gettext
```

```
mingw-w64-x86_64-gettext
├─mingw-w64-x86_64-expat
├─mingw-w64-x86_64-gcc-libs
│ ├─mingw-w64-x86_64-gmp
│ ├─mingw-w64-x86_64-libwinpthread-git provides mingw-w64-x86_64-libwinpthread
│ └─mingw-w64-x86_64-gcc-libgfortran
│   └─mingw-w64-x86_64-gcc-libs
└─mingw-w64-x86_64-libiconv
```

IPSI Lab, Yuntech-CSIE

Method 2. (`pacman -Qi`)

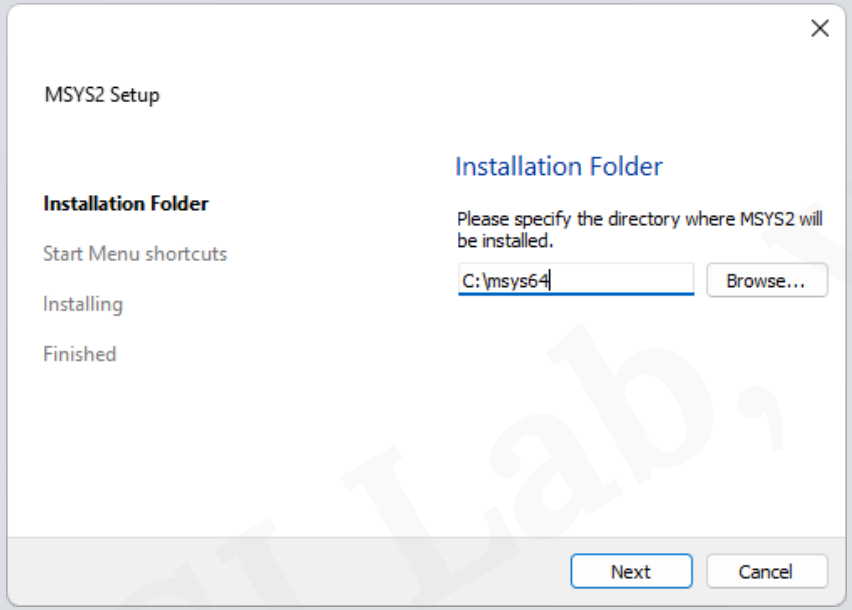
```
$ pacman -Qi mingw-w64-x86_64-gettext
```

```
Name            : mingw-w64-x86_64-gettext
Version         : 0.19.7-1
Description     : GNU internationalization library (mingw-w64)
[...]
Depends On      : mingw-w64-x86_64-expat  mingw-w64-x86_64-gcc-libs
                  mingw-w64-x86_64-libiconv
```

IPSI Lab, Yuntech-CSIE

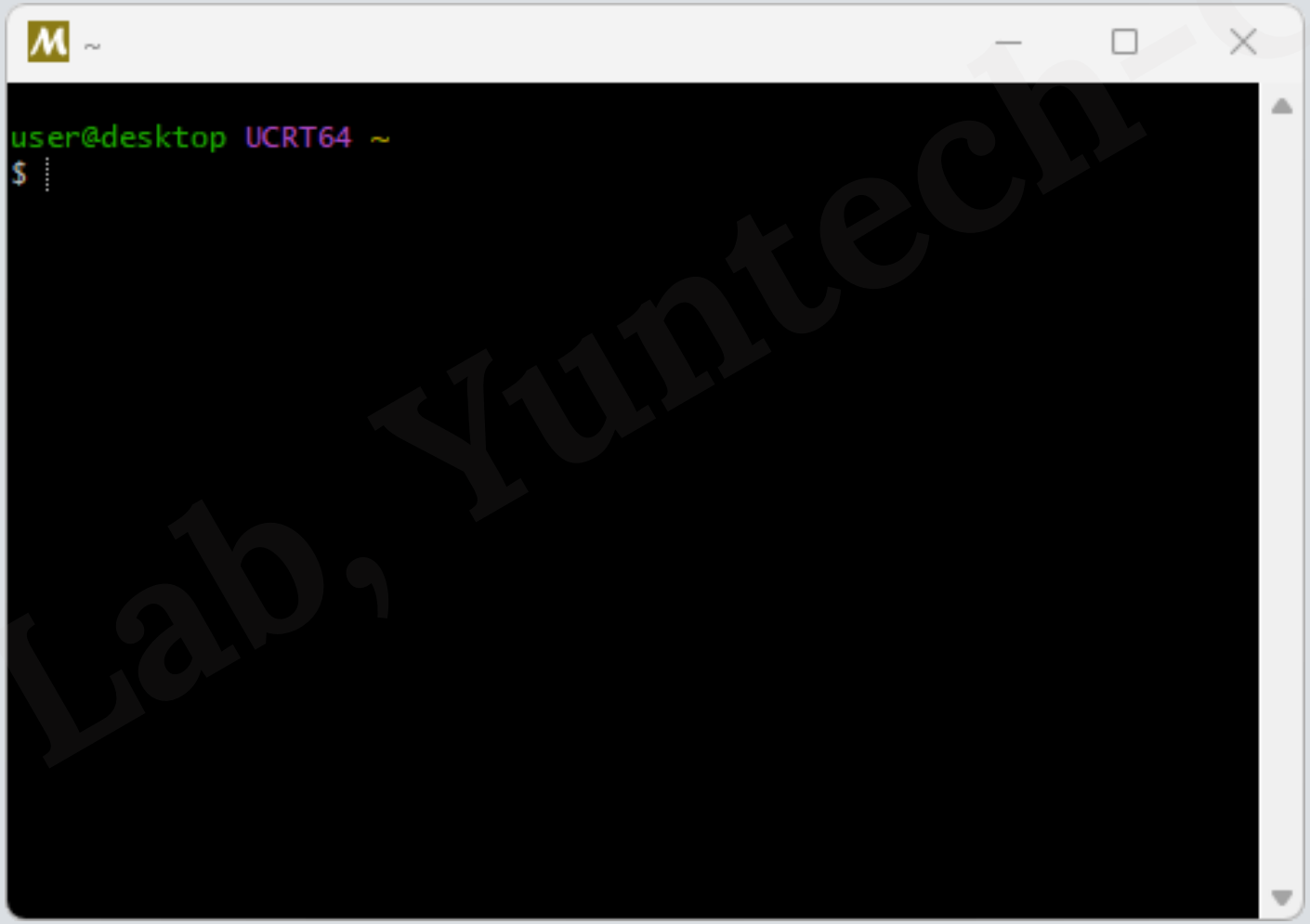
MYSYS2 Installation

Case 1: Installation (Only GCC)

1. Download the installer: https://www.msys2.org/
2. Run the installer. Installing MSYS2 requires 64 bit Windows 10 or newer.
3. 

4. When done, click Finish.

5. Now MSYS2 is ready for you and a terminal for the UCRT64 environment will launch.



IPSI Lab, Yuntech-CSIE

6. You will probably want to install some tools like the mingw-w64 GCC to start compiling projects. Run the following command:

```
$ pacman -S mingw-w64-ucrt-x86_64-gcc
```

IPSI Lab, Yuntech-CSIE

7. The terminal window will show the output as below. Press 'Enter' to continue:

```
resolving dependencies...
looking for conflicting packages...

Packages (15) mingw-w64-ucrt-x86_64-binutils-2.41-2
              mingw-w64-ucrt-x86_64-crt-git-11.0.0.r216.gffe883434-1
              mingw-w64-ucrt-x86_64-gcc-libs-13.2.0-2  mingw-w64-ucrt-x86_64-gmp-6.3.0-2
              mingw-w64-ucrt-x86_64-headers-git-11.0.0.r216.gffe883434-1
              mingw-w64-ucrt-x86_64-isl-0.26-1  mingw-w64-ucrt-x86_64-libiconv-1.17-3
              mingw-w64-ucrt-x86_64-libwinpthread-git-11.0.0.r216.gffe883434-1
              mingw-w64-ucrt-x86_64-mpc-1.3.1-2  mingw-w64-ucrt-x86_64-mpfr-4.2.1-2
              mingw-w64-ucrt-x86_64-windows-default-manifest-6.4-4
              mingw-w64-ucrt-x86_64-winpthreads-git-11.0.0.r216.gffe883434-1
              mingw-w64-ucrt-x86_64-zlib-1.3-1  mingw-w64-ucrt-x86_64-zstd-1.5.5-1
              mingw-w64-ucrt-x86_64-gcc-13.2.0-2

Total Download Size:    49.38 MiB
Total Installed Size:  418.82 MiB

:: Proceed with installation? [Y/n]
[... downloading and installation continues ...]
```

8. Press 'Enter' to continue. Then, reopen MSYS terminal.
9. Now you can call gcc to build software for Windows.

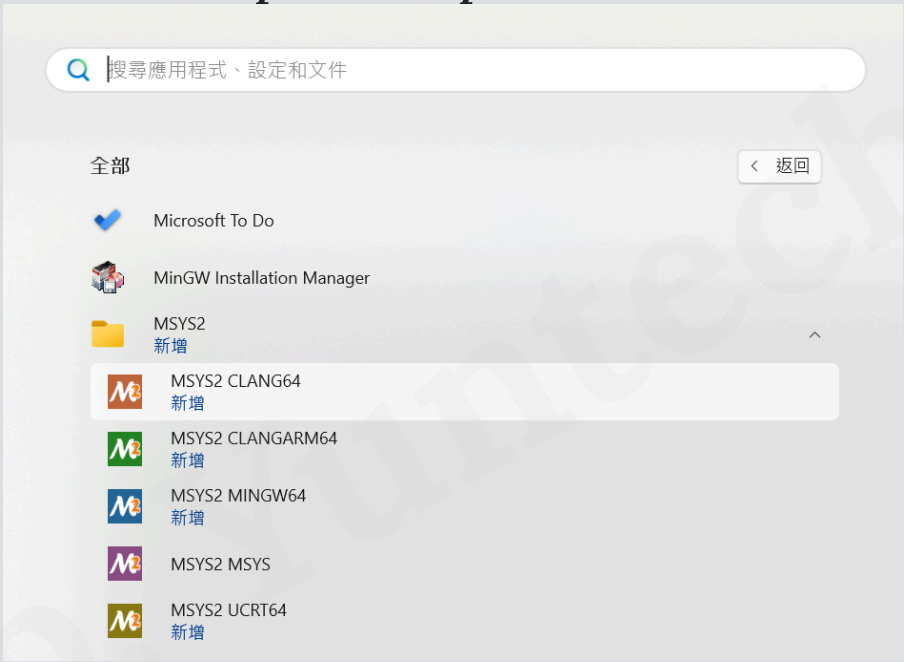
```
$ gcc --version
gcc.exe (Rev2, Built by MSYS2 project) 13.2.0
```

10. After installing MSYS2 it will update itself via pacman.

```
$ pacman -Syu
```

IPSI Lab, Yuntech-CSIE

11. Reopen MSYS Window to Complete Update



Then,

```
$ pacman -Sy
```

12. (option) Used in Windows CMD
    Set environment variables on Windows (環境變數)
    `C:\msys64\mingw64\bin`
    Check GCC version .

```
C:\ > gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=C:/msys64/mingw64/bin/../lib/gcc/x86_64-w64-mingw32/14.1.0/lto-wrapper.exe
Target: x86_64-w64-mingw32
Configured with: (skip...)
Thread model: posix
Supported LTO compression algorithms: zlib zstd
gcc version 14.1.0 (Rev3, Built by MSYS2 project)
```

IPSI Lab, Yuntech-CSIE

Case 2: Installation (Full Toolchain)

```
pacman -S mingw-w64-x86_64-toolchain
```

```
pacman -S --needed < name of the toolchain >
```

- `--needed`：這個選項表示只安裝尚未安裝的軟體包。若這些軟體包已經存在，則跳過，不會重複安裝。
- `base-devel`：這是開發基本工具的集合，通常包括編譯器（如 gcc）、編譯工具（如 make）、以及其他開發相關的工具（如 binutils、patch 等）。

Ref: https://packages.msys2.org/groups/mingw-w64-x86_64-toolchain

- Toolchain
  - `mingw-w64-ucrt-i686-toolchain`
  - `mingw-w64-ucrt-x86_64-toolchain`
  - `mingw-w64-i686-toolchain`
  - `mingw-w64-x86_64-toolchain`

IPSI Lab, Yuntech-CSIE

```
$ pacman -S --needed base-devel mingw-w64-x86_64-toolchain
:: There are 13 members in group mingw-w64-x86_64-toolchain:
:: Repository mingw64
   1) mingw-w64-x86_64-binutils  2) mingw-w64-x86_64-crt-git  3) mingw-w64-x86_64-gcc
   4) mingw-w64-x86_64-gdb  5) mingw-w64-x86_64-gdb-multiarch  6) mingw-w64-x86_64-headers-git
   7) mingw-w64-x86_64-libmangle-git  8) mingw-w64-x86_64-libwinpthread-git
   9) mingw-w64-x86_64-make  10) mingw-w64-x86_64-pkgconf  11) mingw-w64-x86_64-tools-git
   12) mingw-w64-x86_64-winpthreads-git  13) mingw-w64-x86_64-winstorecompat-git

Enter a selection (default=all):
resolving dependencies...
looking for conflicting packages...

Packages (53) binutils-2.42-1  bison-3.8.2-5  diffstat-1.66-1  diffutils-3.10-1  dos2unix-7.5.2-1
              flex-2.6.4-3  m4-1.4.19-2  make-4.4.1-2  mingw-w64-x86_64-bzip2-1.0.8-3
              mingw-w64-x86_64-expat-2.6.2-1  mingw-w64-x86_64-gcc-libs-14.1.0-3
              mingw-w64-x86_64-gettext-runtime-0.22.5-2  mingw-w64-x86_64-gmp-6.3.0-2
              mingw-w64-x86_64-isl-0.26-1  mingw-w64-x86_64-libffi-3.4.6-1
              mingw-w64-x86_64-libiconv-1.17-4  mingw-w64-x86_64-libsystre-1.0.1-5
              mingw-w64-x86_64-libtre-git-r177.07e66d0-2  mingw-w64-x86_64-mpc-1.3.1-2
              mingw-w64-x86_64-mpdecimal-4.0.0-1  mingw-w64-x86_64-mpfr-4.2.1-2
              mingw-w64-x86_64-ncurses-6.4.20231217-1  mingw-w64-x86_64-openssl-3.3.1-1
              mingw-w64-x86_64-python-3.11.9-1  mingw-w64-x86_64-readline-8.2.010-1
              mingw-w64-x86_64-sqlite3-3.46.0-1  mingw-w64-x86_64-tcl-8.6.13-1
              mingw-w64-x86_64-termcap-1.3.1-7  mingw-w64-x86_64-tk-8.6.13-1
              mingw-w64-x86_64-tzdata-2024a-1  mingw-w64-x86_64-windows-default-manifest-6.4-4
              mingw-w64-x86_64-xxhash-0.8.2-2  mingw-w64-x86_64-xz-5.6.2-2
              mingw-w64-x86_64-zlib-1.3.1-1  mingw-w64-x86_64-zstd-1.5.6-2  patch-2.7.6-2
              pkgconf-2.1.1-1  texinfo-7.1-3  texinfo-tex-7.1-3  base-devel-2022.12-2
              mingw-w64-x86_64-binutils-2.42-2  mingw-w64-x86_64-crt-git-12.0.0.r81.g90abf784a-1
              mingw-w64-x86_64-gcc-14.1.0-3  mingw-w64-x86_64-gdb-15.1-1
              mingw-w64-x86_64-gdb-multiarch-15.1-1
              mingw-w64-x86_64-headers-git-12.0.0.r81.g90abf784a-1
              mingw-w64-x86_64-libmangle-git-12.0.0.r81.g90abf784a-1
              mingw-w64-x86_64-libwinpthread-git-12.0.0.r81.g90abf784a-1
              mingw-w64-x86_64-make-4.4.1-2  mingw-w64-x86_64-pkgconf-1~2.2.0-1
              mingw-w64-x86_64-tools-git-12.0.0.r81.g90abf784a-1
              mingw-w64-x86_64-winpthreads-git-12.0.0.r81.g90abf784a-1
              mingw-w64-x86_64-winstorecompat-git-12.0.0.r81.g90abf784a-1

Total Download Size:   126.01 MiB
Total Installed Size:  915.42 MiB

:: Proceed with installation? [Y/n]
```

UNIX 系統路徑格式

絕對路徑 (Absolute Path)

- 從根目錄 `/` 開始，描述檔案或目錄的完整路徑。
- 根目錄 `/` 是整個檔案系統的起點。
- 範例：

```
/home/user/Documents/file.txt
```

- 這表示檔案 `file.txt` 位於 `/home/user/Documents/` 目錄中。

IPSI Lab, Yuntech-CSIE

相對路徑 (Relative Path)

- 基於當前工作目錄，描述檔案或目錄的相對位置。
- 不以 `/` 開頭，而是從當前目錄開始。
- 範例：

```
./Documents/file.txt
```

- 這表示 `file.txt` 在當前目錄的 `Documents` 子目錄中。

IPSI Lab, Yuntech-CSIE

特殊符號

- `.` **(單點)**：代表當前目錄。例如，`./file.txt` 表示當前目錄下的 `file.txt` 檔案。
- `..` **(雙點)**：代表上層目錄。例如，`../file.txt` 表示上層目錄中的 `file.txt` 檔案。
- `~` **(波浪號)**：代表當前使用者的家目錄。例如，`~/Documents` 表示使用者的家目錄下的 `Documents` 目錄。

IPSI Lab, Yuntech-CSIE

路徑中的空格

- 如果路徑或檔案名中有空格，需要用反斜杠 (`\`) 進行轉義，或使用引號。
- 範例：

```
/home/user/My\ Documents/file.txt
# 或
"/home/user/My Documents/file.txt"
```

IPSI Lab, yuntech-CSIE

MSYS2檔案目錄結構

MSYS2 的虛擬檔案系統

- **MSYS2 的檔案系統像 Unix 一樣，有 `/home` 、 `/usr` 、 `/bin` 等典型的 Unix 目錄結構。這些目錄存放的檔案在實際上是存放在 MSYS2 安裝的根目錄下。**
  - **預設路徑**：MSYS2 預設安裝在 `C:\msys64` 下（64 位系統）。
  - **`/home`**：你在 MSYS2 中的家目錄 `/home/user` ，實際存放在 `C:\msys64\home\user` 。
  - **`/usr/bin`**：這些是 MSYS2 的工具和可執行檔案，存放在 `C:\msys64\usr\bin` 。
- **EX. 在 MSYS2 中使用 `/home/user/file.txt` 時，實際上在Windows 系統的 `C:\msys64\home\user\file.txt` 。**

Windows 檔案系統

- **MSYS2 可以直接存取 Windows 的檔案系統，並且將 Windows 的磁碟映射為虛擬路徑。**
  - **C:** 磁碟對應到 `/c/` ，也就是 Windows 的 `C:\` 。
  - **D:** 磁碟對應到 `/d/` ，也就是 Windows 的 `D:\` 。
- **可以在 MSYS2 中直接訪問 Windows 的檔案系統。**
  - Windows 中的 `C:\Users\YourName\Documents\file.txt` ，在 MSYS2 中可以通過 `/c/Users/YourName/Documents/file.txt` 存取。

IPSI Lab, Yuntech-CSIE

Windows CMD VS. MSYS2 (Unix-like)

IPSI Lab, Yuntech-CSIE

路徑格式差異

- Windows CMD 使用的是反斜杠 (`\`) 作為路徑分隔符，而 MSYS2（類似 Unix 的環境）使用的是正斜杠 (`/`)。
  - 在 CMD 中： `C:\Users\YourName`
  - 在 MSYS2 中： `/c/Users/YourName`
- 如果你直接將 Windows 路徑複製到 MSYS2 中，MSYS2 可能無法正確解讀它，尤其是像 `\n` 這樣的路徑會被解讀為換行符號。

IPSI Lab, Yuntech-CSIE

路徑轉換問題

- **自動轉換**：MSYS2 提供了自動的 Windows 路徑與 Unix 路徑的轉換機制，但在某些情況下你需要手動進行轉換，尤其是在將路徑傳遞給不同類型的工具（例如 Windows 工具與 Cygwin 工具之間）時。
- 使用 `cygpath` 工具進行手動轉換。例如：

```
$ cygpath -u "C:\\Program Files"  # Converts Windows to Unix format: /c/Program Files
      $ cygpath -u C:\\foo    # Converts Windows to Unix format: /c/foo
      $ cygpath -m /mingw64/bin  # Converts Unix to Windows: C:/msys64/mingw64/bin
```

- 如果要避免自動轉換，可以設置環境變數 `MSYS2_ENV_CONV_EXCL` ，來指定哪些變數應排除轉換。

環境變數差異

- **Windows CMD** 中的環境變數與 **MSYS2** 中的處理方式有所不同。例如，Windows 中的路徑分隔符是 `;`，而 MSYS2 中是 `:`。這在設置環境變數（例如 `PATH`）時需要注意，特別是當你將 Windows 工具和 MSYS2 工具混合使用時，可能會出現問題。
- **Windows**：

```
set PATH=C:\Program Files\Tool;C:\Windows\System32
```

- **MSYS2**：

```
export PATH=/usr/local/bin:/usr/bin:/c/Program\ Files/Tool
```

指令語法差異

- **Windows CMD** 和 **MSYS2** 使用的命令語法有明顯不同：
  - CMD 使用的命令：`dir`, `copy`, `move`, `del` 等
  - MSYS2 使用 Unix 標準的命令：`ls`, `cp`, `mv`, `rm` 等
- 如果你習慣 CMD 的指令，切換到 MSYS2 需要適應新的 Unix 標準指令。此外，MSYS2 支持 Bash shell 特性，如管道 (`|`) 和文件重定向（`>` 和 `>>`），這些在 CMD 中也存在但語法不同。

IPSI Lab, Yuntech-CSIE

最常用的 UNIX 命令列表：

1. `ls` - 列出目錄內容。
2. `cd` - 切換目錄。
3. `pwd` - 顯示當前目錄。
4. `cp` - 複製檔案。
5. `mv` - 移動或重命名檔案。
6. `rm` - 刪除檔案。
7. `mkdir` - 建立目錄。
8. `cat` - 顯示檔案內容。
9. `echo` - 輸出字串。
10. `chmod` - 修改檔案權限。

IPSI Lab, Yuntech-CSIE

文件系統的大小寫敏感性

- **Windows** 文件系統對文件名大小寫不敏感，而 **MSYS2**（以及 Unix-like 環境）對大小寫敏感。因此，在 MSYS2 中，你需要注意文件名的大小寫。例如，`File.txt` 和 `file.txt` 在 MSYS2 中是兩個不同的文件，而在 CMD 中它們是相同的。

IPSI Lab, Yuntech-CSIE

批次檔與 Shell 腳本

Windows CMD 使用的是 `.bat` 或 `.cmd` 批次檔，而 **MSYS2** 使用的是 `.sh` 的 Shell 腳本。如果你有大量的 `.bat` 檔案，這些無法直接在 MSYS2 中執行，需要轉換成 Shell 腳本格式。

IPSI Lab, Yuntech-CSIE

簡單程式測試

使用 MSYS2 Terminal (`/home/user`)

IPSI Lab, Yuntech-CSIE

1. 撰寫 C 程式

可以使用 MSYS2 中的任何文字編輯器（如 `nano`、`vim` 或 `emacs`）來撰寫 C 程式，或者你也可以使用 Windows 的文字編輯器來編寫程式碼。

用 `nano` 編輯器為例，進入編輯器：

```
nano hello.c
```

IPSI Lab, Yuntech-CSIE

在編輯器中編輯簡單的C程式：

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

完成後按 `Ctrl + X` ，再按 `Y` 來儲存並退出編輯器。
此時檔案存放於 `/home`

IPSI Lab, Yuntech-CSIE

2. 編譯 C 程式

在 MSYS2 終端中使用 GCC 編譯 C 程式。

```
gcc hello.c -o hello
```

此命令將編譯 `hello.c` 並編譯成可執行文件 `hello`。

IPSI Lab, Yuntech-CSIE

3. 執行 C 程式

在 MSYS2 中，路徑遵循 Unix 格式（使用 `/`），所以當執行程式時，要使用 `./` 來表示當前目錄。

```
./hello
# 螢幕上應會顯示： Hello, World!
```

Ipsi Lab, yuntech-CSIE

使用 MSYS2 Terminal (files@anywhere)

如果你的 C 檔案位於 `C:\Users\YourName\Documents\` ，在 MSYS2 中執行以下命令：

```
cd /c/Users/YourName/Documents/
```

或是複製此路徑後，先進行Windows to Unix路徑轉換

```
cygpath -u "C:\Users\YourName\Documents"

# 將得到以下內容
/c/Users/YourName/Documents
```

或是直接執行

```
cd $(cygpath -u "C:\Users\YourName\Documents")
```

IPSI Lab, Yuntech-CSIE

使用 Windows CMD

IPSI Lab, Yuntech-CSIE

1. 務必設定環境變數

2. 撰寫 C 程式

使用任何 Windows 下的文字編輯器來撰寫 C 程式。

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

該檔案儲存為 `hello.c` 。

IPSI Lab, Yuntech-CSIE

3. 編譯 C 程式

開啟 **Command Prompt**（CMD）。

使用 `cd` 命令移動到存放 `hello.c` 檔案的目錄。例如：

```
cd C:\path\to\your\c\program
```

使用 `gcc` 編譯你的 C 程式：

```
gcc hello.c -o hello.exe
```

編譯成 Windows 可執行檔 `hello.exe`。

IPSI Lab, Yuntech-CSIE

4. 執行 C 程式

在 CMD 中直接執行

```
hello.exe
```

你應該會在螢幕上看到：

```
Hello, World!
# 螢幕上應會顯示： Hello, World!
```

IPSI Lab, Yuntech-CSIE

隨堂練習

1. 在 MSYS2 的操作環境下，使用GNU gcc 進行範例程式編譯，以截圖呈現作答，包含以下內容
   - 整個MSYS2環境畫面，也同時清楚寫出是哪一個版本的MSYS2 shell
   - gcc的版本 （包含指令與結果）
   - 程式編譯過程（從下什麼指令到編譯結果整個過程）

IPSI Lab, Yuntech-CSIE